Drawing Board / Whiteboard (Canvas) + Undo/Redo

- **Problem:** Build a drawing board on an HTML canvas. The user draws freehand with a pen, erases, draws straight lines and rectangles, picks a color and a brush size, and can undo, redo and clear (clear is undoable too). Keyboard shortcuts work, the drawing stays sharp on retina screens, survives a window resize, and can be exported as a PNG.
- **Tech stack:** React (Vite, JavaScript, Tailwind CSS v4)
- **Time limit:** 60 minutes
- **Often asked at:** Microsoft, Figma, Miro
- **Why it matters:** Canvas, history stack

PART A: Thought process

General steps for any machine coding task

1. **Understand the requirements.** Restate the problem, split functional vs non-functional, ask clarifying questions, write down assumptions.
2. **List the interactions.** Every user action and its expected result. This is your checklist for building and testing.
3. **Design the architecture.** Name the components and their one-line jobs. Start small.
4. **Design the state.** What is the single source of truth? What can be derived?
5. **Design data structures and contracts.** Shape of the data, component props, reducer actions.
6. **Trace one data flow end to end.** Proves the design works before you type.
7. **Find the hard parts.** Solve them on paper first.
8. **Build in layers.** Simplest working version first, then one working layer at a time.
9. **Cover states and edge cases.** Empty board, nothing to undo, a click without a drag, resize, pointer cancel.
10. **Wrap up.** Trade-offs and what you would add with more time.

Time budget for 60 minutes

Phase	Time
Requirements + clarifying questions	5 min
Interactions, architecture, state design	6 min
Hard parts on paper (history model, render loop, HiDPI)	5 min
Build: canvas + pen drawing with pointer events	10 min
Build: shapes as data + history reducer + undo/redo/clear	12 min
Build: offscreen cache + rAF paint, eraser, line, rectangle	10 min

Phase	Time
Build: HiDPI + resize, shortcuts, export	7 min
Edge cases, accessibility, wrap-up	5 min

Tip: think out loud

Say every decision and its reason as you make it: "I store strokes as data, not pixels, so undo is just popping an array and resize can redraw." The interviewer grades how you think as much as the final code. If you get stuck, say what you are trying and what you would try next.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Freehand **pen** drawing with mouse, pen or touch.
- **Eraser** that removes what is under it.
- **Line** and **rectangle** tools with a live preview while dragging.
- **Color** picker and **brush size** slider.
- **Undo** and **redo** (buttons + `Ctrl/Cmd+Z`, `Ctrl/Cmd+Shift+Z`, `Ctrl+Y`).
- **Clear** the board, and clear can be undone.
- **Export** the drawing as a PNG.

Non-functional requirements

Area	What it means here
Performance	Drawing must keep up with the pointer at 60 fps, even with hundreds of strokes on the board. No React re-render per mouse move.
Scalability	History memory must be bounded (cap the undo stack). Redraw cost must not grow on every frame with the number of shapes.
Error handling	Pointer cancel (touch scroll, alert) drops the stroke cleanly. A click without a drag with line/rect does not create an empty history entry.
Accessibility / usability	Real `<button>`s with `aria-pressed` for the active tool, disabled undo/redo when there is nothing to do, keyboard shortcuts, canvas has `role="img"` and a label with the shape count, `touch-action: none` so the page does not scroll while drawing.
Reusability	`historyReducer` and `drawShape` are pure functions with no React or DOM inside: reusable and testable in Node.
Extensibility	A new tool = one entry in `TOOLS` + one branch in `drawShape`. Persistence or collaboration later = serialize the shapes array.

Clarifying questions to ask the interviewer

1. Which tools do you need: just pen, or also eraser and shapes (line, rectangle)?
2. Is clear expected to be undoable? (It usually is: a user who clears by mistake will hit Ctrl+Z.)
3. Should the drawing survive a refresh (localStorage) or is in-memory enough?
4. Does it need to work on touch devices and with a stylus (pressure)?
5. Is the canvas a fixed size or should it fill the screen and handle resize?
6. Do you want selection / moving shapes after they are drawn, or is it a paint-style board?

Assumptions we will build with

- Tools: pen, eraser, line, rectangle. One color and one size per shape.
- Undo/redo works on whole shapes (one stroke = one step), and clear is one undoable step.
- History is capped at 100 steps.
- In-memory only (persistence is a "more time" item).
- Canvas fills the space under the toolbar and redraws on resize. Shapes keep their position in CSS pixels; they are not scaled.
- No selection or moving shapes (that turns it into a vector editor, a different problem).

2. Identify interactions / use cases

Action / input	Expected behavior / output
Pointer down + move with Pen	A smooth line follows the pointer in the chosen color and size
Pointer up	The stroke becomes one shape in history; Undo enables, Redo disables
Click (no drag) with Pen	A dot is drawn and committed
Drag with Line / Rectangle	A live preview follows the pointer; on release the shape is committed
Click (no drag) with Line / Rectangle	Nothing is drawn, no history entry
Drag with Eraser	Pixels under the eraser disappear; it is one undoable step
Click Undo / press Ctrl+Z	Last step is removed from the board; Redo enables
Click Redo / Ctrl+Shift+Z / Ctrl+Y	The undone step comes back
Draw something new after Undo	The redo branch is discarded, Redo disables
Click Clear	Board becomes empty; Undo brings the whole picture back
Undo with empty history / Redo with empty future	Nothing happens (buttons are disabled)
Change color or size	Only new shapes use it; existing shapes keep theirs
Resize the window	Canvas fills the new area and the drawing is redrawn, still sharp
Pointer leaves the canvas mid-stroke	The stroke keeps going (pointer capture) and commits on release
Pointer cancel (touch scroll, alert)	The in-progress stroke is dropped
Click Export PNG	Downloads `drawing.png` with a white background

3. Design the architecture

Module	Responsibility
`App`	Owns the history (`useReducer`) and tool settings, wires shortcuts and export.
`Toolbar`	Pure UI: tool buttons, color, size, undo/redo/clear/export. Calls callbacks, holds no state.
`Board`	Owns the canvas: pointer handling, the live draft, the offscreen cache, HiDPI and resize.
`utils/history.js`	Pure `historyReducer` : commit / undo / redo / clear on `{ past, present, future }` .
`utils/draw.js`	Pure `drawShape(ctx, shape)` , the tool list, and `isDegenerate` .

```
App  (history: { past, present, future }, tool, color, size)
├── Toolbar   props: tool, color, size, canUndo, canRedo, canClear, callbacks
└── Board     props: shapes = history.present, tool, color, size, onCommit
      ├── <canvas>            visible layer
      ├── cache canvas        (offscreen, committed shapes)
      └── draftRef            (shape being drawn, not React state)
utils/history.js   used by App
utils/draw.js      used by Board (and Toolbar for TOOLS)
```

Start simple: one component with a canvas and a pen. Split `Toolbar` out as soon as the buttons appear, and move the reducer and drawing into `utils` as soon as they are pure, so you can test them.

4. Design the state / data model

State or field	Purpose
`history.past`	Stack of earlier snapshots (arrays of shapes), newest last. Capped at 100.
`history.present`	The shapes currently on the board. This is what gets drawn.
`history.future`	Snapshots that were undone, nearest first. Emptied on any new action.
`tool`	Active tool: `'pen' | 'eraser' | 'line' | 'rect'` .
`color` , `size`	Brush settings used for the next shape.
`draftRef` (ref)	The shape being drawn right now. Mutated on every pointer move.
`cacheRef` (ref)	Offscreen canvas with all committed shapes already painted.
`box` (state in Board)	`{ w, h, dpr }` of the drawing area, set by `ResizeObserver` .

```
export const initialHistory = { past: [], present: [], future: [] };

const [history, dispatch] = useReducer(historyReducer, initialHistory);
const [tool, setTool] = useState('pen');
```

```
const [color, setColor] = useState('#1f2937');
const [size, setSize] = useState(4);
```

Senior note:

- **Derived, not stored:** `canUndo = past.length > 0`, `canRedo = future.length > 0`, `canClear = present.length > 0`, the canvas aria label. Never store these as separate flags; they would get out of sync.
- **The pixels are derived too.** The canvas is a view of `present`. The single source of truth is the shapes array, never the bitmap.
- **Ownership:** `App` owns history and settings because both the toolbar and the board need them. `Board` owns everything that is about pixels (draft, cache, size), because nobody else needs it.
- **Why the draft is a ref, not state:** pointer move fires 60-240 times a second. Putting points into state would re-render React on each of them. The draft only needs to reach the canvas, so a ref + `requestAnimationFrame` is enough. It enters React state once, on pointer up.

5. Design the data structures and contracts

Shape (one history step)

```
// Freehand: every point. Line / rect: [start, end] only.
{ id: '9b1c…', tool: 'pen',    color: '#1f2937', size: 4,  points: [{ x: 120, y: 80 }, { x: 124, y: 83 }, …] }
{ id: '3f0a…', tool: 'eraser', color: '#1f2937', size: 20, points: [{ x: 300, y: 200 }, …] }
{ id: 'c7d2…', tool: 'rect',   color: '#dc2626', size: 2,  points: [{ x: 40, y: 40 }, { x: 200, y: 120 }] }
```

- `id` from `crypto.randomUUID()`: a stable key for later features (select, delete one shape, sync with a server).
- Points are in **CSS pixels**, not device pixels, so the data does not depend on the screen's `devicePixelRatio`.
- Each shape carries its own `color` and `size`, so changing the brush never changes old shapes.

History snapshot

```
{
  past:    [[], [s1]],          // states before
  present: [s1, s2],            // what is on screen
  future:  [],                  // undone states
}
```

Snapshots share shape objects: `[s1]` and `[s1, s2]` point to the same `s1`. A snapshot costs one array of references, not a copy of every point.

Reducer contract

Action	Effect
`{ type: 'commit', shape }`	push `present` to `past`, `present = [...present, shape]`, `future = []`
`{ type: 'clear' }`	same push with `present = []`; no-op if already empty
`{ type: 'undo' }`	`present` goes to the front of `future`, last of `past` becomes `present`
`{ type: 'redo' }`	`present` goes to `past`, first of `future` becomes `present`
no-op case	returns the **same object**, so React skips the re-render

Component contracts

```
<Board canvasRef shapes tool color size onCommit={(shape) => …} />
<Toolbar tool onToolChange color onColorChange size onSizeChange
         canUndo canRedo canClear onUndo onRedo onClear onExport />
drawShape(ctx, shape)       // pure: data -> pixels, no React
historyReducer(state, action) -> state
```

There is no server API. If the interviewer asks for one, the contract is the shapes array as JSON: `GET /boards/:id -> { shapes }`, `POST /boards/:id/shapes { shape }`.

6. Trace the data flow

One pen stroke, then Undo:

```
pointerdown on canvas
  -> setPointerCapture, draftRef = { id, tool:'pen', color, size, points:[p0] }
  -> schedulePaint()                      (one rAF)
pointermove (many times)
  -> push coalesced points into draftRef.points   (no React render)
  -> schedulePaint()                      (still at most one paint per frame)
rAF fires -> paint()
  -> clear visible canvas
  -> drawImage(cache)                     (all committed shapes, one call)
  -> drawShape(ctx, draft)                (just the live stroke)
pointerup
  -> draftRef = null, onCommit(draft)
  -> dispatch({ type:'commit', shape })    -> new history.present
  -> Board effect [shapes] rebuilds cache: clear + drawShape for each shape
  -> paint() -> stroke now lives in the cache
Ctrl+Z
  -> dispatch({ type:'undo' }) -> present = previous snapshot
  -> same effect rebuilds cache without the stroke -> paint()
  -> Toolbar: Redo enabled (future.length > 0)
```

7. Important technical challenges

Challenge 1: What does history store? Pixels or shapes?

Problem: The naive undo saves `canvas.toDataURL()` or `getImageData()` after every stroke. A 1600x1000 canvas on a retina screen is about 6.4 MB of RGBA per snapshot; 100 undo steps is 640 MB. Resize also breaks: a saved bitmap is the old size.

Solution: Store **shapes as data** (vector). The picture is `present.map(drawShape)`. Undo, redo and clear just move arrays between three stacks. Snapshots share shape objects, so each one costs an array of pointers. Clear is a normal step (`present = []`), so it is undoable for free. Any new action empties `future` (the standard "new edit kills the redo branch" rule), and `past` is capped.

```
function push(state, present) {
  return { past: [...state.past, state.present].slice(-MAX_HISTORY), present, future: [] };
}

case 'commit': return push(state, [...state.present, action.shape]);
case 'clear':  return state.present.length ? push(state, []) : state;
case 'undo':   if (!state.past.length) return state;
               return { past: state.past.slice(0, -1), present: state.past.at(-1),
                        future: [state.present, ...state.future] };
```

The eraser fits the same model: it is a stroke drawn with `globalCompositeOperation = 'destination-out'`, so it removes pixels when replayed and can be undone like any other shape.

Challenge 2: Drawing fast without re-rendering React or redrawing everything

Problem: If each `pointermove` calls `setState`, React re-renders 120+ times a second. If each frame replays all shapes, a board with 500 strokes of 300 points drops frames. A rectangle preview also needs the old frame erased every move, so you cannot just draw on top.

Solution: Two layers:

- An **offscreen cache canvas** holds every committed shape. It is rebuilt only when `shapes` (or the size) changes: on commit, undo, redo, clear, resize.
- The **live draft** lives in a ref. Pointer moves mutate it and request one `requestAnimationFrame`. Each frame: clear, `drawImage(cache)` (one GPU blit), draw the draft on top.

Per frame cost is O(draft), not O(all shapes). The preview of a rectangle or line is free: the frame is rebuilt from the cache each time.

```
const paint = () => {
  ctx.setTransform(1, 0, 0, 1, 0, 0);
  ctx.clearRect(0, 0, canvas.width, canvas.height);
  ctx.drawImage(cacheRef.current, 0, 0);
  if (draftRef.current) {
    ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
    drawShape(ctx, draftRef.current);
```

```
  }
};

const schedulePaint = () => {
  if (frameRef.current) return;                 // already one paint queued this frame
  frameRef.current = requestAnimationFrame(() => { frameRef.current = 0; paint(); });
};
```

The eraser preview also works here: drawn with `destination-out` on the visible canvas, it cuts through the cache pixels just blitted.

Challenge 3: Blurry lines on retina screens, and resize wiping the canvas

Problem: A canvas sized only with CSS is a 300x150 bitmap stretched to fit: lines are blurry. On a `devicePixelRatio = 2` screen, a 800x500 CSS canvas needs a 1600x1000 bitmap. And setting `canvas.width` (which you must do on resize) **wipes the canvas**.

Solution: A `ResizeObserver` stores `{ w, h, dpr }`. The effect sets the bitmap size to `w * dpr` by `h * dpr` (only when it changed), scales the context with `setTransform(dpr, …)`, and redraws from the shapes array. Because the truth is data, a wipe costs nothing: we redraw. Points stay in CSS pixels, so the mouse math never needs `dpr`.

```
useEffect(() => {
  if (!box) return;
  const { w, h, dpr } = box;
  const W = Math.round(w * dpr), H = Math.round(h * dpr);
  for (const c of [canvas, cache]) if (c.width !== W || c.height !== H) { c.width = W; c.height = H; }
  ctx.setTransform(1, 0, 0, 1, 0, 0);
  ctx.clearRect(0, 0, W, H);
  ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
  shapes.forEach((s) => drawShape(ctx, s));
  paint();
}, [shapes, box]);
```

Challenge 4: Smooth strokes and reliable pointer input

Problem: Fast mouse moves give points 20-40 px apart, so `lineTo` between them looks like a polygon. The browser also merges several moves into one event per frame, dropping points. If the pointer leaves the canvas, `pointermove` stops and the stroke is cut. On touch, dragging scrolls the page instead of drawing.

Solution:

- `getCoalescedEvents()` returns the merged points, so we keep all of them.
- Draw **quadratic curves through midpoints** instead of straight segments.
- `setPointerCapture` keeps events coming outside the canvas; `pointercancel` drops the stroke.
- CSS `touch-action: none` (`touch-none`) stops the page scrolling. Pointer events cover mouse, touch and pen with one code path.

```
const events = e.nativeEvent.getCoalescedEvents?.() ?? [];
(events.length ? events : [e]).forEach((ev) => draft.points.push(toPoint(ev, rect)));

ctx.moveTo(a.x, a.y);
for (let i = 1; i < points.length - 1; i++) {
  const p = points[i], q = points[i + 1];
  ctx.quadraticCurveTo(p.x, p.y, (p.x + q.x) / 2, (p.y + q.y) / 2);
}
ctx.lineTo(last.x, last.y);
```

8. Implement in the correct order

#	Step	Working result after it
1	Layout: title, toolbar row, a canvas that fills the rest	Static page
2	Pointer down/move/up drawing straight onto the canvas with `lineTo`	You can scribble (no undo yet)
3	Store each stroke as a shape object; draw from `shapes` state on change	Same look, but the drawing is now data
4	`historyReducer` with commit/undo/redo + buttons	Undo and redo work
5	Clear as a history step; disabled states for buttons	Undoable clear
6	Move the draft into a ref + rAF; add the offscreen cache	Smooth drawing with many shapes
7	Line and rectangle tools with live preview; skip zero-size shapes	Shape tools
8	Eraser with `destination-out`	Undoable eraser
9	`ResizeObserver` + `devicePixelRatio` scaling	Sharp and responsive
10	Pointer capture, coalesced events, curve smoothing, `touch-none`	Feels like a real drawing app
11	Keyboard shortcuts, export PNG, aria attributes	Done
12	Node self-check for the reducer and `drawShape`	Proven logic

9. Cover all states and edge cases

State / edge case	Handling
Initial / empty board	"Click and drag to draw" hint; Undo, Redo, Clear, Export disabled
Drawing in progress	Draft painted per frame; not in history until pointer up
After a commit	Undo enabled, Redo disabled (redo branch dropped)
Nothing to undo / redo	Buttons disabled; reducer returns the same state (no re-render) for shortcuts
Undo then draw	`future` emptied, Redo disabled
Clear on empty board	No-op, no extra history entry

State / edge case	Handling
Clear then Undo	Whole picture comes back
History over 100 steps	Oldest snapshot dropped; the board still shows every shape
Single click with pen	Dot via `arc` (a zero-length path is not drawn reliably)
Single click with line / rect	`isDegenerate` drops it; no empty history step
Pointer leaves canvas mid-stroke	Pointer capture keeps the stroke going
`pointercancel`	Draft dropped, frame repainted
Right / middle click	Ignored (`e.button !== 0`)
Window resize / phone rotation	Bitmap resized, redrawn from data, still sharp
Retina screen	Bitmap scaled by `devicePixelRatio`
Touch device	`touch-action: none` , pointer events, no page scroll
Eraser export	Erased pixels are transparent, so export paints white underneath
Color picker while eraser selected	Disabled: the eraser ignores color
Undo pressed mid-stroke	Board updates under the draft; the draft still commits on release

10. Final structure

```
_78_drawing_board_undo_redo/
├── index.html
├── vite.config.js
├── package.json
└── src/
    ├── main.jsx
    ├── index.css
    ├── App.jsx                history + settings + shortcuts + export
    ├── components/
    │   ├── Toolbar.jsx        tools, color, size, undo/redo/clear/export
    │   └── Board.jsx          canvas, pointer input, cache, HiDPI, resize
    └── utils/
        ├── history.js         pure undo/redo reducer
        └── draw.js            TOOLS, drawShape, isDegenerate
check.mjs                      node assert self-check (outside src)
```

11. Full working code

Create the project with `npm create vite@latest` (React, JavaScript), then `npm i -D tailwindcss @tailwindcss/vite` .

vite.config.js

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react-swc';
import tailwindcss from '@tailwindcss/vite';

export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Drawing Board</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

src/index.css

```
@import "tailwindcss";
```

src/main.jsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import './index.css';
import App from './App';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

`src/utils/history.js`

```
// Undo/redo as three stacks of snapshots. A snapshot is an array of shape
// references, so shapes are shared between snapshots, never copied.
export const MAX_HISTORY = 100;

export const initialHistory = { past: [], present: [], future: [] };

// Every new action pushes the current picture to past and kills the redo branch.
function push(state, present) {
  return {
    past: [...state.past, state.present].slice(-MAX_HISTORY),
    present,
    future: [],
  };
}

export function historyReducer(state, action) {
  switch (action.type) {
    case 'commit':
      return push(state, [...state.present, action.shape]);
    case 'clear':
      return state.present.length ? push(state, []) : state;
    case 'undo': {
      if (!state.past.length) return state;
      return {
        past: state.past.slice(0, -1),
        present: state.past.at(-1),
        future: [state.present, ...state.future],
      };
    }
    case 'redo': {
      if (!state.future.length) return state;
      return {
        past: [...state.past, state.present],
        present: state.future[0],
        future: state.future.slice(1),
      };
    }
    default:
      return state;
  }
}
```

`src/utils/draw.js`

```
export const TOOLS = [
  { id: 'pen', label: 'Pen' },
  { id: 'eraser', label: 'Eraser' },
  { id: 'line', label: 'Line' },
  { id: 'rect', label: 'Rectangle' },
];

// Freehand tools collect every point; line/rect only keep [start, end].
export const FREEHAND = new Set(['pen', 'eraser']);

// A click with the line or rectangle tool draws nothing, so it must not create a history entry.
export function isDegenerate({ tool, points }) {
  if (FREEHAND.has(tool)) return false;
  const [a, b] = points;
  return a.x === b.x && a.y === b.y;
}

// Pure "data -> pixels". The same function paints committed shapes and the live draft.
export function drawShape(ctx, { tool, color, size, points }) {
  ctx.save();
  // Eraser is a normal stroke that removes pixels, so it is undoable like any other shape.
  ctx.globalCompositeOperation = tool === 'eraser' ? 'destination-out' : 'source-over';
  ctx.strokeStyle = color;
  ctx.fillStyle = color;
  ctx.lineWidth = size;
  ctx.lineCap = 'round';
  ctx.lineJoin = 'round';

  const [a, b] = points;
  if (tool === 'rect') {
    ctx.strokeRect(a.x, a.y, b.x - a.x, b.y - a.y);
  } else if (points.length === 1) {
    // A single click with the pen leaves a dot.
    ctx.beginPath();
    ctx.arc(a.x, a.y, size / 2, 0, Math.PI * 2);
    ctx.fill();
  } else {
    // Curve through the midpoints so fast strokes look smooth instead of jagged.
    ctx.beginPath();
    ctx.moveTo(a.x, a.y);
    for (let i = 1; i < points.length - 1; i++) {
      const p = points[i];
      const q = points[i + 1];
      ctx.quadraticCurveTo(p.x, p.y, (p.x + q.x) / 2, (p.y + q.y) / 2);
    }
```

```
    const last = points.at(-1);
    ctx.lineTo(last.x, last.y);
    ctx.stroke();
  }
  ctx.restore();
}
```

src/components/Board.jsx

```
import { useEffect, useRef, useState } from 'react';
import { drawShape, FREEHAND, isDegenerate } from '../utils/draw';

export default function Board({ canvasRef, shapes, tool, color, size, onCommit }) {
  const wrapRef = useRef(null);
  const cacheRef = useRef(null); // offscreen canvas with every committed shape already painted
  const draftRef = useRef(null); // shape being drawn right now; kept out of React state
  const boxRef = useRef(null);
  const frameRef = useRef(0);
  const [box, setBox] = useState(null); // { w, h, dpr } of the drawing area

  // Track the container size so the canvas always fills it.
  useEffect(() => {
    const ro = new ResizeObserver(([entry]) => {
      const { width, height } = entry.contentRect;
      setBox({ w: Math.round(width), h: Math.round(height), dpr: window.devicePixelRatio || 1 });
    });
    ro.observe(wrapRef.current);
    return () => {
      ro.disconnect();
      cancelAnimationFrame(frameRef.current);
    };
  }, []);

  // Visible canvas = cached committed layer (one drawImage) + the live draft on top.
  const paint = () => {
    const canvas = canvasRef.current;
    if (!canvas || !cacheRef.current || !boxRef.current) return;
    const ctx = canvas.getContext('2d');
    ctx.setTransform(1, 0, 0, 1, 0, 0);
    ctx.clearRect(0, 0, canvas.width, canvas.height);
    ctx.drawImage(cacheRef.current, 0, 0);
    if (draftRef.current) {
      const { dpr } = boxRef.current;
      ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
      drawShape(ctx, draftRef.current);
    }
  };
```

```
// Many pointermove events per frame -> at most one paint per frame.
const schedulePaint = () => {
  if (frameRef.current) return;
  frameRef.current = requestAnimationFrame(() => {
    frameRef.current = 0;
    paint();
  });
};

// Rebuild the cache only when the committed shapes (undo/redo/commit/clear) or the size change.
useEffect(() => {
  if (!box) return;
  boxRef.current = box;
  const { w, h, dpr } = box;
  const W = Math.round(w * dpr);
  const H = Math.round(h * dpr);
  const canvas = canvasRef.current;
  const cache = (cacheRef.current ??= document.createElement('canvas'));
  for (const c of [canvas, cache]) {
    // Setting width/height wipes a canvas, so only do it when the size really changed.
    if (c.width !== W || c.height !== H) {
      c.width = W;
      c.height = H;
    }
  }
  const ctx = cache.getContext('2d');
  ctx.setTransform(1, 0, 0, 1, 0, 0);
  ctx.clearRect(0, 0, W, H);
  // Draw in CSS pixels; the dpr scale keeps lines sharp on retina screens.
  ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
  shapes.forEach((shape) => drawShape(ctx, shape));
  paint();
  // paint only reads refs, so it does not need to be a dependency.
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [shapes, box]);

const toPoint = (e, rect) => ({ x: e.clientX - rect.left, y: e.clientY - rect.top });

const handlePointerDown = (e) => {
  if (e.button !== 0) return;
  // Keep receiving moves even when the pointer leaves the canvas mid-stroke.
  e.currentTarget.setPointerCapture(e.pointerId);
  const p = toPoint(e, e.currentTarget.getBoundingClientRect());
  draftRef.current = {
    id: crypto.randomUUID(),
    tool,
```

```
      color,
      size,
      points: FREEHAND.has(tool) ? [p] : [p, p],
    };
    schedulePaint();
  };

  const handlePointerMove = (e) => {
    const draft = draftRef.current;
    if (!draft) return;
    const rect = e.currentTarget.getBoundingClientRect();
    if (FREEHAND.has(draft.tool)) {
      // Coalesced events give the points the browser merged between frames: smoother fast strokes.
      const events = e.nativeEvent.getCoalescedEvents?.() ?? [];
      (events.length ? events : [e]).forEach((ev) => draft.points.push(toPoint(ev, rect)));
    } else {
      draft.points[1] = toPoint(e, rect);
    }
    schedulePaint();
  };

  const handlePointerUp = () => {
    const draft = draftRef.current;
    if (!draft) return;
    draftRef.current = null;
    if (isDegenerate(draft)) schedulePaint();
    else onCommit(draft); // new shapes -> effect rebuilds the cache and repaints
  };

  // Browser took the pointer away (e.g. a touch scroll or alert): drop the stroke.
  const handlePointerCancel = () => {
    draftRef.current = null;
    schedulePaint();
  };

  return (
    <div ref={wrapRef} className="relative min-h-0 flex-1 overflow-hidden rounded-xl border border-
slate-200 bg-white shadow-sm">
      <canvas
        ref={canvasRef}
        role="img"
        aria-label={`Drawing canvas with ${shapes.length} shape${shapes.length === 1 ? '' : 's'}`}
        className="absolute inset-0 h-full w-full cursor-crosshair touch-none"
        onPointerDown={handlePointerDown}
        onPointerMove={handlePointerMove}
        onPointerUp={handlePointerUp}
        onPointerCancel={handlePointerCancel}
```

```
      />
      {shapes.length === 0 && (
        <p className="pointer-events-none absolute inset-0 grid place-items-center text-sm text-slate-
400">
          Click and drag to draw
        </p>
      )}
    </div>
  );
}
```

src/components/Toolbar.jsx

```
import { TOOLS } from '../utils/draw';

const btn =
  'rounded-lg border px-3 py-1.5 text-sm font-medium transition-colors focus-visible:outline-2 focus-
visible:outline-offset-2 focus-visible:outline-indigo-500 disabled:cursor-not-allowed disabled:opacity-
40';

export default function Toolbar({
  tool,
  onToolChange,
  color,
  onColorChange,
  size,
  onSizeChange,
  canUndo,
  canRedo,
  canClear,
  onUndo,
  onRedo,
  onClear,
  onExport,
}) {
  return (
    <div role="toolbar" aria-label="Drawing tools" className="flex flex-wrap items-center gap-3">
      <div className="flex gap-1">
        {TOOLS.map((t) => (
          <button
            key={t.id}
            type="button"
            aria-pressed={tool === t.id}
            onClick={() => onToolChange(t.id)}
            className={`${btn} ${
              tool === t.id
                ? 'border-indigo-600 bg-indigo-600 text-white'
```

```
                : 'border-slate-300 bg-white text-slate-700 hover:bg-slate-100'
            }`}
          >
            {t.label}
          </button>
        ))}
      </div>

      <label className="flex items-center gap-2 text-sm text-slate-700">
        Color
        <input
          type="color"
          value={color}
          onChange={(e) => onColorChange(e.target.value)}
          disabled={tool === 'eraser'}
          className="h-8 w-10 cursor-pointer rounded border border-slate-300 disabled:opacity-40"
        />
      </label>

      <label className="flex items-center gap-2 text-sm text-slate-700">
        Size
        <input
          type="range"
          min="1"
          max="40"
          value={size}
          onChange={(e) => onSizeChange(Number(e.target.value))}
        />
        <span className="w-6 tabular-nums">{size}</span>
      </label>

      <div className="ml-auto flex gap-1">
        <button type="button" onClick={onUndo} disabled={!canUndo} title="Undo (Ctrl+Z)" className=
{`${btn} border-slate-300 bg-white text-slate-700 hover:bg-slate-100`}>
          Undo
        </button>
        <button type="button" onClick={onRedo} disabled={!canRedo} title="Redo (Ctrl+Shift+Z / Ctrl+Y)"
className={`${btn} border-slate-300 bg-white text-slate-700 hover:bg-slate-100`}>
          Redo
        </button>
        <button type="button" onClick={onClear} disabled={!canClear} className={`${btn} border-red-300
bg-white text-red-600 hover:bg-red-50`}>
          Clear
        </button>
        <button type="button" onClick={onExport} disabled={!canClear} className={`${btn} border-slate-
300 bg-white text-slate-700 hover:bg-slate-100`}>
          Export PNG
```

```
        </button>
      </div>
    </div>
  );
}
```

src/App.jsx

```
import { useEffect, useReducer, useRef, useState } from 'react';
import Board from './components/Board';
import Toolbar from './components/Toolbar';
import { historyReducer, initialHistory } from './utils/history';

export default function App() {
  const [history, dispatch] = useReducer(historyReducer, initialHistory);
  const [tool, setTool] = useState('pen');
  const [color, setColor] = useState('#1f2937');
  const [size, setSize] = useState(4);
  const canvasRef = useRef(null);

  // Global shortcuts: Ctrl/Cmd+Z undo, Ctrl/Cmd+Shift+Z or Ctrl+Y redo.
  useEffect(() => {
    const onKeyDown = (e) => {
      if (!(e.ctrlKey || e.metaKey)) return;
      const key = e.key.toLowerCase();
      if (key === 'z') {
        e.preventDefault();
        dispatch({ type: e.shiftKey ? 'redo' : 'undo' });
      } else if (key === 'y') {
        e.preventDefault();
        dispatch({ type: 'redo' });
      }
    };
    window.addEventListener('keydown', onKeyDown);
    return () => window.removeEventListener('keydown', onKeyDown);
  }, []);

  // Erased pixels are transparent, so paint a white background under the drawing before export.
  const exportPng = () => {
    const src = canvasRef.current;
    const out = document.createElement('canvas');
    out.width = src.width;
    out.height = src.height;
    const ctx = out.getContext('2d');
    ctx.fillStyle = '#ffffff';
    ctx.fillRect(0, 0, out.width, out.height);
    ctx.drawImage(src, 0, 0);
```

```
    const a = document.createElement('a');
    a.href = out.toDataURL('image/png');
    a.download = 'drawing.png';
    a.click();
  };

  return (
    <main className="flex h-screen flex-col gap-3 bg-slate-100 p-4">
      <h1 className="text-lg font-semibold text-slate-800">Drawing board</h1>
      <Toolbar
        tool={tool}
        onToolChange={setTool}
        color={color}
        onColorChange={setColor}
        size={size}
        onSizeChange={setSize}
        canUndo={history.past.length > 0}
        canRedo={history.future.length > 0}
        canClear={history.present.length > 0}
        onUndo={() => dispatch({ type: 'undo' })}
        onRedo={() => dispatch({ type: 'redo' })}
        onClear={() => dispatch({ type: 'clear' })}
        onExport={exportPng}
      />
      <Board
        canvasRef={canvasRef}
        shapes={history.present}
        tool={tool}
        color={color}
        size={size}
        onCommit={(shape) => dispatch({ type: 'commit', shape })}
      />
    </main>
  );
}
```

`check.mjs` (run with `node check.mjs`)

```
import assert from 'node:assert/strict';
import { historyReducer, initialHistory, MAX_HISTORY } from './app/src/utils/history.js';
import { drawShape, isDegenerate } from './app/src/utils/draw.js';

const shape = (id) => ({ id, tool: 'pen', color: '#000', size: 2, points: [{ x: 0, y: 0 }] });
const run = (actions, s = initialHistory) => actions.reduce(historyReducer, s);

// commit, undo, redo
let s = run([{ type: 'commit', shape: shape('a') }, { type: 'commit', shape: shape('b') }]);
```

```
assert.deepEqual(s.present.map((x) => x.id), ['a', 'b']);
s = historyReducer(s, { type: 'undo' });
assert.deepEqual(s.present.map((x) => x.id), ['a']);
assert.equal(s.future.length, 1);
s = historyReducer(s, { type: 'redo' });
assert.deepEqual(s.present.map((x) => x.id), ['a', 'b']);

// new commit after undo kills the redo branch
s = run([{ type: 'undo' }, { type: 'commit', shape: shape('c') }], s);
assert.deepEqual(s.present.map((x) => x.id), ['a', 'c']);
assert.equal(s.future.length, 0);

// clear is undoable; clearing an empty board is a no-op
s = historyReducer(s, { type: 'clear' });
assert.deepEqual(s.present, []);
assert.equal(historyReducer(s, { type: 'clear' }), s);
s = historyReducer(s, { type: 'undo' });
assert.deepEqual(s.present.map((x) => x.id), ['a', 'c']);

// undo/redo at the ends and unknown actions return the same object (no re-render)
assert.equal(historyReducer(initialHistory, { type: 'undo' }), initialHistory);
assert.equal(historyReducer(initialHistory, { type: 'redo' }), initialHistory);
assert.equal(historyReducer(s, { type: 'nope' }), s);

// shapes are shared between snapshots, not copied
assert.equal(s.past.at(-1)[0], s.present[0]);

// history is capped
let big = initialHistory;
for (let i = 0; i < MAX_HISTORY + 20; i++) big = historyReducer(big, { type: 'commit', shape:
shape(String(i)) });
assert.equal(big.past.length, MAX_HISTORY);
assert.equal(big.present.length, MAX_HISTORY + 20);

// zero-size line/rect is dropped; pen click is kept
assert.equal(isDegenerate({ tool: 'rect', points: [{ x: 1, y: 1 }, { x: 1, y: 1 }] }), true);
assert.equal(isDegenerate({ tool: 'line', points: [{ x: 1, y: 1 }, { x: 2, y: 1 }] }), false);
assert.equal(isDegenerate({ tool: 'pen', points: [{ x: 1, y: 1 }] }), false);

// drawShape: fake context records calls
const fakeCtx = () => {
  const calls = [];
  const ctx = new Proxy({ calls }, {
    get: (t, k) => (k in t ? t[k] : (...args) => calls.push([k, ...args])),
    set: (t, k, v) => ((t[k] = v), true),
  });
  return ctx;
```

```
};
let ctx = fakeCtx();
drawShape(ctx, shape('dot'));
assert.ok(ctx.calls.some(([m]) => m === 'arc'));
ctx = fakeCtx();
drawShape(ctx, { tool: 'eraser', color: '#000', size: 10, points: [{ x: 0, y: 0 }, { x: 5, y: 5 }, { x:
9, y: 9 }] });
assert.equal(ctx.globalCompositeOperation, 'destination-out');
assert.deepEqual(ctx.calls.map(([m]) => m), ['save', 'beginPath', 'moveTo', 'quadraticCurveTo',
'lineTo', 'stroke', 'restore']);
ctx = fakeCtx();
drawShape(ctx, { tool: 'rect', color: '#f00', size: 2, points: [{ x: 10, y: 10 }, { x: 4, y: 30 }] });
assert.deepEqual(ctx.calls.find(([m]) => m === 'strokeRect'), ['strokeRect', 10, 10, -6, 20]);

console.log('all checks passed');
```

12. Wrap-up

Edge cases handled

- Undo/redo at the ends of history, redo branch dropped after a new edit, undoable clear, capped history.
- Click without drag (dot for pen, nothing for line/rect), pointer cancel, pointer leaving the canvas, non-left buttons.
- Retina sharpness, resize without losing the drawing, touch without page scroll.
- Eraser that is undoable and exports with a white background.

Trade-offs to mention

- **Snapshots vs command pattern.** Snapshots (`past/present/future` of arrays) are the simplest correct model and make clear trivial. The cost is an O(n) array copy per step. A command stack (`{ do, undo }` per action, e.g. `add shape` / `remove shape`) is O(1) per step and is what you would pick for a big editor or for collaboration.
- **Vector vs bitmap.** Storing shapes keeps undo cheap and resize lossless, but the cache rebuild replays every shape on each undo. For tens of thousands of strokes you would keep periodic bitmap checkpoints and replay only from the nearest one.
- **Eraser as pixels, not object delete.** `destination-out` matches paint apps (it can erase half a stroke). A Figma/Miro style "object eraser" would delete whole shapes by hit testing instead.
- **Shapes stay in CSS pixels on resize.** Simple and predictable; drawings are not scaled to the new size (a fixed "world" with pan/zoom would fix that).
- `devicePixelRatio` **read on resize only.** Moving the window to a monitor with a different DPR does not re-scale until the next resize.

What to add with more time

- Persist `present` to `localStorage` (debounced) or a backend; load on start.
- Pressure-sensitive pen width with `e.pressure`, and stroke simplification (Ramer-Douglas-Peucker) to shrink point arrays.

- Selection, move, delete of single shapes (hit testing), more shapes, text, fill color.
- Pan and zoom with a view transform; infinite canvas.
- Real-time collaboration: send shape ops over a WebSocket, per-user undo, CRDT for ordering.
- Playwright tests for drawing and shortcuts; screenshot tests of the canvas.
- A live region that announces "Undone" / "Redone" for screen reader users.

13. Counter questions the interviewer may ask

Requirements

Q1. Why canvas and not SVG?

SVG keeps a DOM node per shape: easy hit testing and CSS, but thousands of freehand paths with hundreds of points each get slow to lay out and re-render. Canvas is immediate mode: one bitmap, cost depends on pixels, not on node count. For a paint-style board canvas wins; for a diagram tool with selectable objects SVG (or canvas + your own hit testing) is reasonable.

Q2. What if they want the drawing to survive a refresh?

The truth is already a plain array of shapes, so persistence is `localStorage.setItem('board', JSON.stringify(present))` in a debounced effect and reading it as the initial `present`. History itself is usually not persisted. Watch the 5 MB `localStorage` limit with long strokes; use IndexedDB beyond that.

State design

Q3. Why `useReducer` and not a few `useState` calls?

Undo touches `past`, `present` and `future` together. With three `useState`s you can end up with them out of sync between renders. One reducer makes every transition atomic, pure and testable in Node without React.

Q4. Why not keep the points of the current stroke in state?

Pointer move fires up to 240 times a second on some devices. Every `setState` would re-render `App`, `Toolbar` and `Board`. The draft only feeds the canvas, so it lives in a ref and is painted on `requestAnimationFrame`. It enters React state once, on pointer up.

Q5. You mutate `draft.points` in place. Is that safe?

Yes, because the object is not in React state while it is mutated. On pointer up we set `draftRef.current = null` and hand the object to the reducer; nobody mutates it after that. If we mutated a shape that was already in `present`, old snapshots would change too and undo would break.

Challenge 1: history model

Q6. How much memory do 100 snapshots use?

Each snapshot is an array of references to shared shape objects. With 500 shapes that is 100 x 500 pointers, roughly 400 KB, while the points are stored once. Pixel snapshots of a retina canvas would be about 6 MB each.

Q7. Why does a new stroke after undo clear the redo stack?

Redo means "replay what I undid on top of this exact state". After a new edit the state is different, so the old future no longer applies. Keeping it would need a history tree (like Vim's undo tree), which users rarely expect.

Q8. How would you implement undo with the command pattern instead?

Each action is an object with `do()` and `undo()`: `AddShape` pushes and pops a shape, `Clear` stores the removed shapes and puts them back. Undo calls `undo()` on the top command and moves it to the redo stack. It is O(1) memory per step and maps directly onto operations you can send to a server for collaboration.

Q9. What if a user draws 10,000 strokes? Undo gets slow, why and how to fix it?

Undo rebuilds the cache by replaying every shape, so it is O(total points). Fix: keep a bitmap checkpoint every N steps (e.g. an `ImageBitmap` every 50 shapes) and replay only the shapes after the nearest checkpoint. Or, for an append-only undo, keep a cache of "all shapes but the last" so a single undo is one blit.

Challenge 2: rendering

Q10. Why `requestAnimationFrame` and not paint inside `pointermove`?

A mouse at 120 Hz on a 60 Hz screen fires two moves per frame; painting both wastes work you never see. rAF batches them into one paint aligned with the display refresh. The flag in `frameRef` makes sure only one frame is queued.

Q11. Why an offscreen canvas instead of two stacked `<canvas>` elements?

Both work. Two stacked canvases (base + overlay) avoid the per-frame `drawImage`, but the eraser preview breaks: erasing on the overlay cannot remove pixels from the base. One visible canvas fed by an offscreen cache keeps the eraser preview correct and a single code path for every tool. The blit is a cheap GPU copy.

Q12. Could you move drawing off the main thread?

Yes, with `canvas.transferControlToOffscreen()` and an `OffscreenCanvas` in a Web Worker: post points to the worker, it paints. It helps when the main thread is busy with other React work. For this app the main-thread cost is already tiny, so it is not worth the complexity.

Challenge 3: HiDPI and resize

Q13. Why do you store points in CSS pixels and not canvas pixels?

`clientX - rect.left` is already in CSS pixels, and the data stays independent of the screen. The `setTransform(dpr, …)` maps CSS pixels to device pixels at draw time. If you stored device pixels, the same drawing would appear at a different size on a non-retina screen.

Q14. Why not just set `canvas.width` on every render?

Setting `width` or `height`, even to the same value, clears the bitmap and resets the context state. That is why the effect only assigns when the size really changed, and why redraw-from-data is required after a resize anyway.

Challenge 4: input

Q15. Why pointer events and not mouse + touch events?

Pointer events give one API for mouse, touch and stylus, with `pointerId`, `pressure` and pointer capture. With mouse + touch you write two code paths and have to stop the emulated mouse events that follow a touch.

Q16. What happens with two fingers on a touch screen?

Each finger gets its own `pointerId`. This code keeps one draft, so the second finger replaces it and both fingers' moves feed the same stroke (a zigzag). To support it properly, key drafts by `pointerId` in a `Map`, or ignore pointers that are not `isPrimary`. Many apps use two fingers for pan/zoom instead.

Implementation steps

Q17. How do you make Ctrl+Z not fire inside a text input?

Check `e.target`: skip when it is an `input`, `textarea` or `contentEditable` element, so the browser's own text undo still works there. Our toolbar only has color and range inputs, where Ctrl+Z means nothing, so the global listener is fine here.

Q18. How does export work, and why the white background?

`canvas.toDataURL('image/png')` reads the bitmap. Erased areas are transparent (`destination-out`), which would show as black or checkered in many viewers, so we copy onto a temporary canvas filled with white first. For big canvases prefer `toBlob` + `URL.createObjectURL` to avoid a huge base64 string.

Performance and scale

Q19. A stroke has thousands of points. How do you shrink it?

Run Ramer-Douglas-Peucker on pointer up: drop points that are within ~0.5 px of the line between their neighbours. It often removes 70-90% of points with no visible change, which shrinks memory, redraw time and network payloads.

Q20. What if the canvas has to be 10,000 x 10,000 (infinite whiteboard)?

Do not allocate a bitmap that big (browsers cap canvas size and memory). Keep a viewport-sized canvas and a camera `{ x, y, zoom }`; draw only shapes whose bounding box intersects the view (spatial index like a grid or R-tree). Shapes live in world coordinates; the pointer is converted with the inverse camera transform.

Trade-offs and "what if"

Q21. How would you make it collaborative like Miro?

Send operations, not pixels: `{ type: 'add', shape }`, `{ type: 'delete', id }` over a WebSocket. Shape ids make ops idempotent. Undo becomes per user: undo only your own ops, by sending their inverse. Ordering conflicts are solved with a server sequence number or a CRDT (Yjs / Automerge).

Q22. Why is the eraser a stroke and not "delete shapes it touches"?

It is a product choice. A pixel eraser lets users erase part of a stroke, like a paint app, and fits the replay model with one compositing flag. An object eraser needs hit testing against every shape but keeps the data clean (no invisible eraser strokes piling up). Ask which one they expect.

Q23. How would you test this?

Unit test the pure parts in Node: the reducer (commit, undo, redo, clear, cap, no-op identity) and `drawShape` with a fake context that records calls. End-to-end with Playwright: drag on the canvas, then check the shape count in the aria label, button disabled states, and a pixel or screenshot comparison.